DuplexSLA: A Full-Duplex Spoken Language Model with Synchronized Speech, Language, and Action

Haoyang Zhang[1,2,3,*], Jun Chen[1,*], Donghang Wu[3], Yuxin Li[1,3]
Yuxin Zhang[1,4], Xiangyu Tony Zhang[1,5], Che Liu[1,6], Qingjian Lin[1]
Yizhou Peng[3], Hexin Liu[3], Eng Siong Chng[3], Chao Yan[1]
Boyong Wu[1], Yechang Huang[1], Xuerui Yang[1], Fei Tian[1,†]
[1]StepFun [2]Peking University [3]Nanyang Technological University
[4]Shanghai Jiao Tong University [5]University of New South Wales [6]Imperial College London
[*]Equal contribution. [†]Corresponding author.

Abstract

Recent advances in spoken dialogue language models have shifted from turn-based to full-duplex designs, where the model continuously listens to the user while generating responses. However, existing duplex backbones still lack a native channel for in-conversation planning and tool calling, leaving real-time agentic behaviour either tied to turn boundaries or relegated to an external cascade. We propose **DuplexSLA**, a native full-duplex *Speech–Language–Action* foundation model that decodes assistant audio together with a structured action stream on a shared 160 ms chunk timeline. DuplexSLA is built on a dual-stream three-channel formulation – a continuous user audio channel, a discrete assistant audio channel, and a rate-limited textual action channel – all decoded jointly by a single backbone, so that listening, speaking, planning, and tool calling unfold on one shared clock. Two capabilities define the model: (1) semantic-driven turn-taking control, where interruption, pause, and backchannel are handled inside the same backbone instead of by an external semantic VAD; and (2) in-conversation planning and tool calling, where planning text and structured tool calls are emitted on the action channel without halting assistant audio, so that multi-action and backchannel-triggered tool use are interleaved with ongoing speech. To evaluate these capabilities together, we further construct **DuplexSLA-Bench**, a duplex benchmark covering pause, interrupt, and backchannel turn-taking together with three styles of in-conversation tool calling. Model weights and the evaluation code will be released at https://github.com/hyzhang24/DuplexSLA.

1 Introduction

Natural conversation is not a strict alternation between two speakers. Listeners begin to plan a response before a turn ends, tolerate hesitations, send short feedback such as "mm-hmm" without taking the floor, and recover quickly from accidental overlap. They also *act* while talking: opening an application or triggering a control on the same conversational clock as their words. A spoken assistant that lacks any of these behaviours feels rigid no matter how strong its underlying language model is.

Most deployed speech agents still rely on a turn-based pipeline of VAD, ASR, LLM, and TTS, which encounters two structural problems in duplex spoken interaction. First, an energy-based VAD cannot distinguish an end-of-turn silence from a hesitation pause, a brief backchannel, or a real interruption; bolting an external semantic VAD on top recovers part of this nuance but adds latency and still does not see the assistant's internal state. We find that integrating these decisions into a native full-duplex backbone is much more effective than externally attaching another semantic VAD: a sufficiently large duplex model with adequate data absorbs pause, backchannel, and interruption phenomena into its core conversational competence, rather than paying the latency of an extra detector chain. Second, tool calling does not fit cleanly into a turn-based loop. Emitting tool calls before the assistant speaks adds wall-clock delay; emitting them after the assistant finishes delays the side-effect by a full turn; and emitting them mid-utterance through the same channel that drives speech tends to break the spoken response. What is missing is a model that can listen, speak, think, and act on *one* synchronized timeline. Recent work has therefore moved toward native full-duplex speech models that learn to listen and speak inside a single backbone [1–14] and toward chunk-aligned reasoning on the same timeline as audio [15, 16], with audio-aware foundation models [17–28] and existing duplex and spoken-language benchmarks [29–39] as supporting context. We focus on a combination that existing duplex backbones and benchmarks do not jointly stress: semantic-driven turn-taking control *plus* in-conversation tool calling.

We propose **DuplexSLA**, a native full-duplex foundation model with a dual-stream three-channel formulation. The model continuously consumes user audio, decodes assistant audio, and emits textual action tokens in lockstep on a shared 160 ms chunk grid. Each chunk carries (1) two 80 ms causal user audio features, (2) four 40 ms discrete assistant audio tokens preceded by a text anchor (a TA4 layout), and (3) up to ten action text tokens that may contain delayed transcript text, planning text, control labels (`interrupt`, `backchannel`, `response`), and structured tool calls. The same backbone autoregressively predicts the assistant TA4 stream and the action stream, while the user audio side is kept causal and is never produced by the model. The two highlight capabilities of DuplexSLA are: (1) semantic-driven native interruption, pause, and backchannel, where chunk-level turn-taking decisions live on the action channel of the same backbone that drives assistant speech, removing the external semantic VAD; and (2) in-conversation planning and tool calling, where planning text and JSON-style tool calls are emitted on the action channel while the assistant TA4 channel keeps producing speech, all under a strict per-chunk action-token budget (§ 2).

The action channel is what makes the third letter of *Speech–Language–Action* non-trivial: it gives every action object and VAD-like decision (`interrupt`, `backchannel`, `response`) a dedicated, time-stamped textual lane co-decoded with assistant audio, instead of either competing for slots in the assistant text channel or being relegated to a post-hoc cascade.

The contributions of this report are as follows.

- We propose **DuplexSLA** (§ 2), a native full-duplex foundation model that co-decodes assistant audio and a structured action stream on a shared chunk timeline.
- We construct **DuplexSLA-Bench** (§ 5), a 2,100-case duplex evaluation suite with a timing-aware tool-call protocol, on which DuplexSLA reaches sub-second latency while remaining competitive on tool-call accuracy.

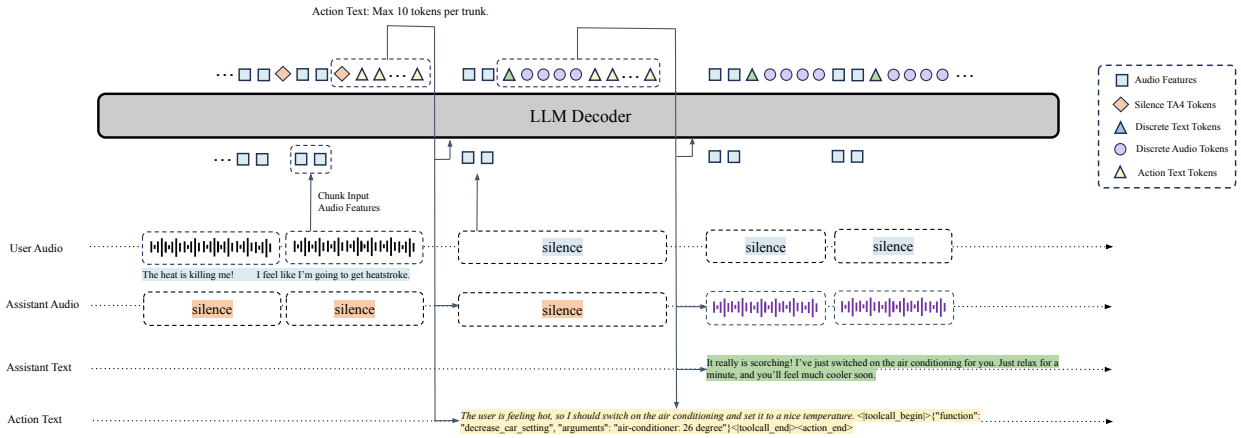

Figure 1: **DuplexSLA chunk-level architecture.** Each chunk is 160 ms. The user channel contributes 2 causal audio features (80 ms each); the assistant channel contributes a TA4 unit (one text anchor and 4 discrete audio tokens at 40 ms each); the action channel emits up to 10 text tokens that may be delayed transcript text, planning text, or tool calls. The same backbone autoregressively predicts the assistant TA4 and the action text.

2 Model Architecture

2.1 Dual-stream three-channel formulation

DuplexSLA models a spoken interaction as a sequence of fixed-size chunks. Let $\Delta = 160$ ms be the chunk size on the conversational clock, with time discretized as $c = \lfloor t/\Delta \rfloor$. At each chunk c, the model receives one user audio segment and one assistant audio segment, and produces two outputs: an assistant audio segment and an action segment, both indexed by c.

We organize each chunk into three channels:

- **User Channel**: a continuous user audio feature sequence. Each chunk contributes 2 causal features at an 80 ms stride.
- **Assistant Channel**: a discrete assistant speech sequence in *TA4* layout. Each chunk emits one text anchor token T followed by 4 audio tokens A at a 40 ms stride.
- **Action Channel**: a textual stream that may contain delayed transcript text, planning text, interaction-control labels, or structured tool calls. The action stream is rate-limited (see § 2.3).

We refer to this design as the dual-stream three-channel formulation: there are two physical audio streams (user and assistant) on the conversational clock, but three semantic channels on the model interface, because the action channel is text-only and lives on top of the assistant timeline. As illustrated in Figure 1, the LLM decoder consumes user audio features together with previously generated assistant audio and action text, and produces the next chunk's assistant audio plus action text in a single autoregressive step.

2.2 Per-chunk serialization

Within a chunk, the three channels are interleaved into a single token stream consumed by the LLM backbone. The serialization is:

<|user_audio_begin|>	$U\ U$	<|user_audio_end|>
<|assistant_audio_begin|>	$T\ A\ A\ A\ A$	<|assistant_audio_end|>
⟨action text⟩		<|action_end|>

The user audio segment is encoded by a causal speech front end (so the streaming property is preserved), while the assistant TA4 unit and the action text are the parts the model must *produce*. Whenever the chunk has nothing to say, T is predicted as a special anchor token (<vad_silence> or <tts_pad>) and the four A tokens are predicted as the corresponding silence audio codes. The <|action_end|> marker terminates the chunk regardless of whether any action text was emitted, which keeps every chunk strictly aligned to the 160 ms clock.

For the action channel, we use a small set of structured markers in addition to free text:

- planning text: a short rationale fragment;
- turn-taking labels: response / interrupt / backchannel;
- tool call: a JSON body wrapped in <|toolcall_begin|>...<|toolcall_end|> that names a function and its arguments.

The action segment for a chunk that carries planning text plus a single tool call therefore has the following abstract form:

```
planning<|toolcall_begin|>{"function": "function_name", "arguments": "arguments"}<|toolcall_end|>
```

Here planning is a free-form rationale fragment, and the JSON body inside <|toolcall_begin|>/<|toolcall_end|> carries the function name and structured arguments. Both the action text and the assistant TA4 unit produced in chunk c are aligned to chunk c on the conversational clock, so a tool call emitted while the assistant is still speaking can be assigned a precise time stamp by reading the chunk index. Compared with backbones that work with two streams only (user and assistant), this explicit separation pushes planning and tool calling onto a dedicated, time-stamped channel without disturbing the assistant audio.

2.3 Real-time decoding budget

Real-time full-duplex interaction requires the per-chunk decoding cost of the model to fit inside one 160 ms chunk on the actual inference hardware. After the assistant TA4 unit is paid for, the autoregressive decoding throughput of a 7B-scale backbone on mainstream inference accelerators leaves room for only a small number of action-channel tokens per chunk. We therefore cap the action channel at 10 text tokens per chunk, with a safe margin against the per-chunk wall-clock budget; tokens that do not fit spill into the action segments of the following chunks. This bound is a deployment budget, not an architectural constraint, and can be re-tuned per accelerator without retraining.

2.4 Native interruption, pause and backchannel

Because the action channel shares a backbone with assistant speech generation, control decisions can be derived from the same internal representation that drives the response.

Three semantic phenomena therefore become intrinsic model behaviours rather than external rules:

- **Pause**: the user holds their thought without ending the turn, and the assistant should remain silent. The action stream stays at `response`-style continue-listening labels, and the assistant TA4 keeps emitting silence anchors.
- **Interrupt**: the user starts a new thought while the assistant is speaking. Around the semantic interruption point, DuplexSLA emits an `interrupt` label on the action channel and switches the assistant TA4 to silence within a small number of chunks.
- **Backchannel**: the user produces a short feedback utterance (e.g., "yes", "you are right") without intending to take the floor. The action channel emits a `backchannel` label, and the assistant continues without resetting its current speech plan.

Crucially, all three decisions are made based on the model's internal semantic state, rather than on a separate VAD module. The benefit is concrete: a turn-based pipeline that bolts a semantic VAD on top still has to pay the latency of the external detector and is bounded by what that detector can express. With sufficient duplex data, a large native duplex model integrates these decisions directly into its token-level dynamics, as reflected in the timing numbers reported in § 5. Figure 2 illustrates the two intuitive cases: a backchannel that does not stop the assistant and an interruption that does.

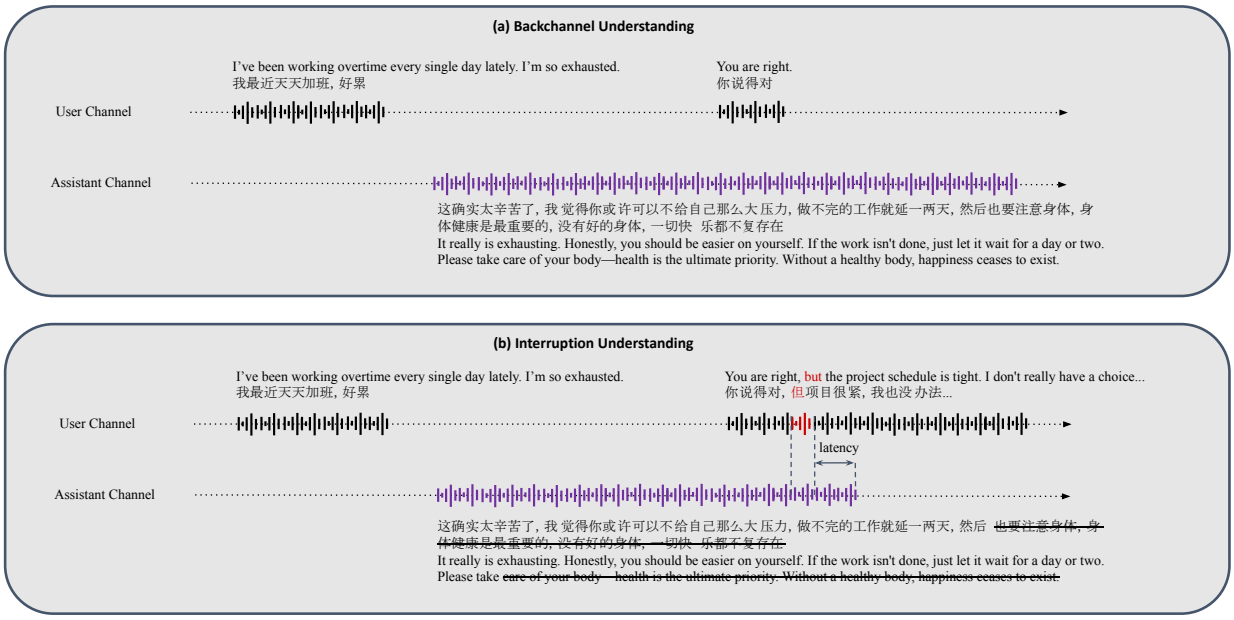

Figure 2: **Native interaction-control behaviours.** (a) A short user backchannel ("You are right") does not stop the assistant; the action channel emits a backchannel label while assistant speech keeps flowing. (b) When the user starts a real new thought ("You are right, but the project schedule is tight…"), DuplexSLA emits an interrupt label and the assistant yields the floor within a small chunk-level latency.

2.5 In-conversation planning and tool calling

The action channel also carries planning text and structured tool calls, which is what keeps tool calling *on-line* rather than a turn-final dump. Two patterns are particularly important.

Backchannel-triggered tool calling. A short user utterance that is topically unrelated to the current dialogue (e.g., "play some Beatles songs" uttered while the assistant is talking about something else) is treated as a backchannel: the action channel emits a planning fragment plus a tool call (`play_music("The Beatles")`) without interrupting the assistant's spoken thread. The assistant audio thus stays coherent while the side-effect is dispatched.

Multi-action tool calling. A single user request can spawn several tool calls, e.g., raising the AC, playing music, and navigating to a restaurant. Because each tool call is anchored to its own chunk on the action channel, the calls are emitted in semantic order along the user's request, and the assistant audio runs in parallel with each call's planning text. This pattern is difficult to express cleanly in turn-based agents, where multi-tool plans either delay all speech until the tools resolve or break the spoken response.

Figure 3 shows both patterns. In the first row, the user issues a backchannel-style request and the action channel emits a tool call without disturbing the assistant. In the second row, a single user turn produces three time-aligned tool calls, each anchored to the relevant chunk on the action channel.

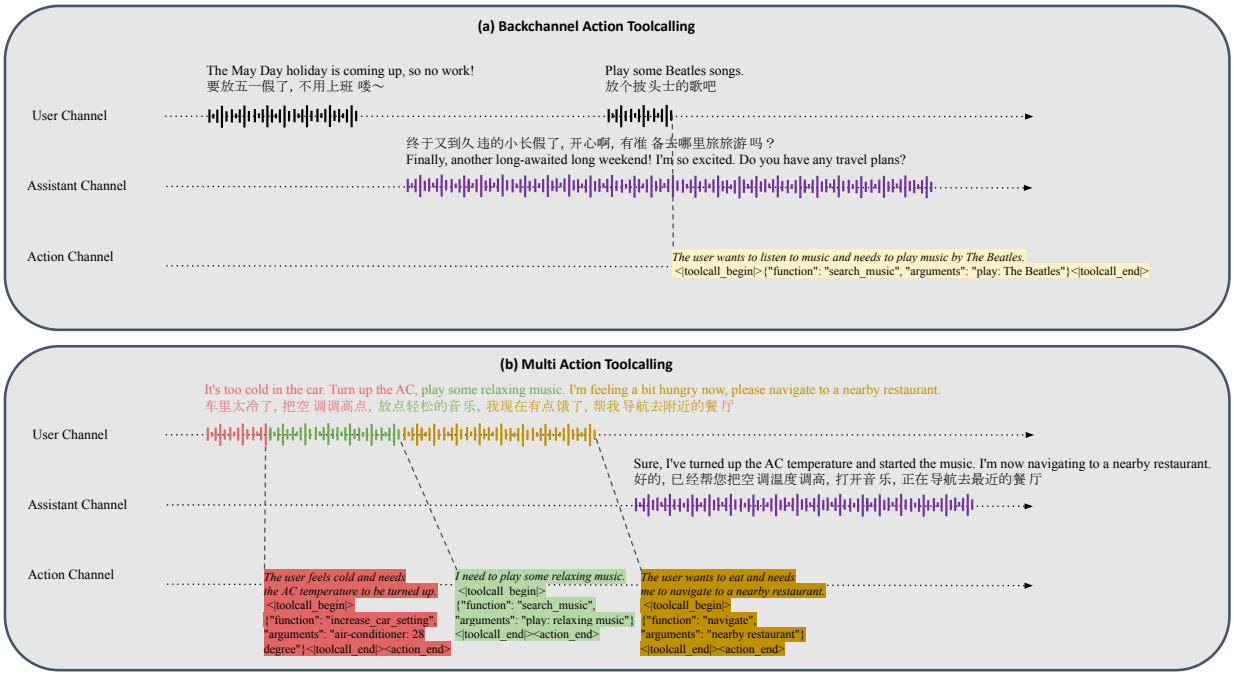

Figure 3: **Planning + tool-call integration on the action channel.** (a) Backchannel-triggered tool call: a brief user utterance triggers `play_music("The Beatles")` on the action channel while the assistant keeps speaking. (b) Multi-action tool calling: one user turn with three intents emits three time-aligned tool calls (AC, music, navigation) on the action channel; assistant speech and action emission run on the same chunk timeline.

2.6 Action channel design rationale

The dedicated third channel is what makes the two highlight capabilities cheap to learn and to serve. Embedding planning and tool calls into the same channel as assistant text would force that channel to alternate between TA4 audio tokens and tool-call JSON, which breaks the smoothness of the assistant audio. A separate action channel that is itself synchronized

to the chunk clock also gives every action object an unambiguous timestamp, which is needed both for downstream execution and for the latency-oriented evaluation in § 5. The cost of the third channel is modest – at most 10 tokens per chunk – and is easily absorbed by the per-chunk decoding budget in § 2.3.

A condensed view of the system is given in Table 1.

Design element	Current formulation
Backbone scale	7B speech-LM, initialized from Step-Audio 2 mini [20]
Streaming clock	160 ms conversational chunks
User audio granularity	2 causal acoustic features per chunk (80 ms each)
Assistant audio granularity	4 discrete audio tokens per chunk (40 ms each)
Per-chunk speech layout	TA4 (one text anchor + four audio tokens)
Action channel content	Delayed transcript text, planning text, turn-taking labels, tool calls
Per-chunk action token budget	≤ 10 tokens; overflow spills into next chunks
Tool-call schema	50 cabin and smart-home function schemas, plus 3 interaction-control labels
Native duplex behaviours	Pause, interrupt, and backchannel without external semantic VAD
Online tool calling	Backchannel-triggered, single-action, multi-action

Table 1: System-level summary of DuplexSLA.

3 Data Construction

The chunked, dual-stream three-channel format described in § 2 does not match the format of conventional dialogue corpora, so building DuplexSLA required a dedicated data-construction effort. The goal of this section is to give a clear picture of *the data form* fed to the model and *the mixture proportions* used in training, rather than to enumerate every annotation detail. Figure 4 summarises the pipeline in two stages.

3.1 Sample form

Every training sample is a chunked dual-track session. The shared schema is as follows:

- a task-conditioned system prompt (one of `dialogue`, `asr_human`, `asr_assistant`, `interrupt`, `backchannel`, `pause`, `toolcall`);
- a user audio track and an assistant audio track aligned on the same conversational clock; the assistant audio is represented as discrete speech units (4 per chunk in TA4 layout);
- an ordered list of action objects, each with a function name, optional planning text, optional structured arguments, and a semantic trigger offset that is snapped to a chunk index at training time;
- the assistant TA4 stream and the action stream are both supervised, while the user audio side is observed only.

The same schema covers all task families: they differ only in which channels carry information. Specifically, ASR families use the action channel for delayed transcript text, timing-control families use it for the `interrupt`, `backchannel`, and `response` labels, and tool-use families use it for planning text plus structured tool calls. A more detailed enumeration of the action vocabulary is given in Appendix B, and full per-task chunk-by-chunk traces in Appendix A.

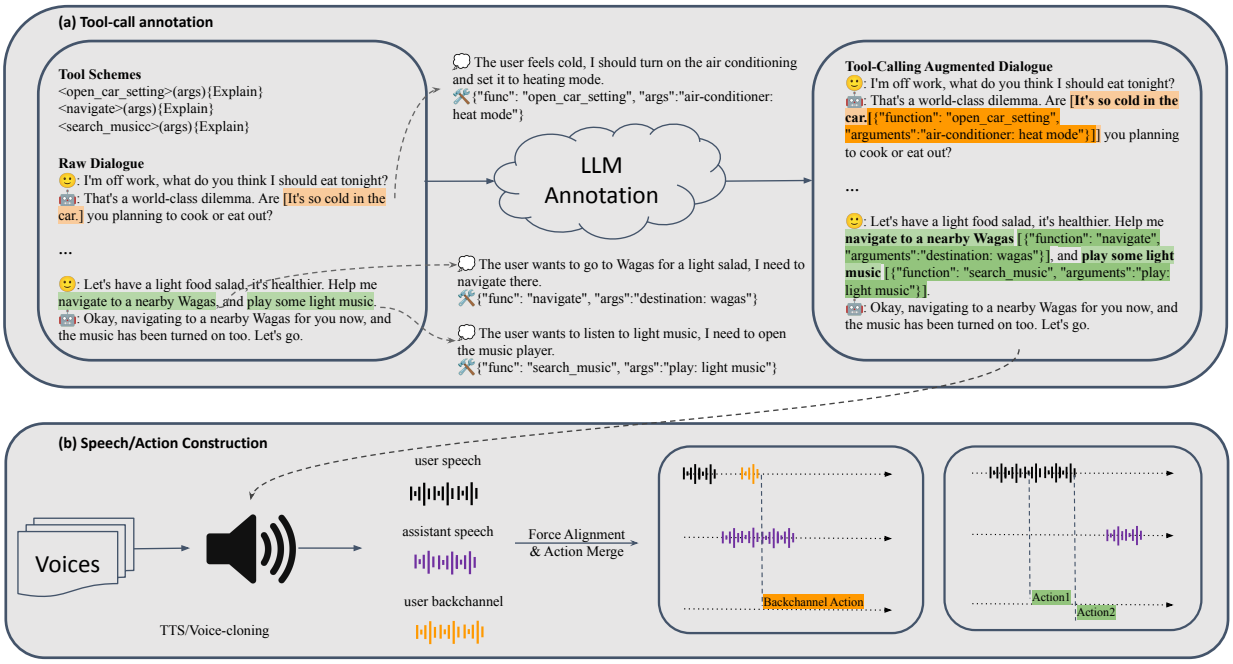

Figure 4: **Data-construction pipeline.** (a) An LLM annotates each raw dialogue with tool-call objects (function name, arguments, planning text, semantic offset). (b) The user and assistant utterances are synthesized with TTS and voice cloning, force-aligned, time-merged, and the action-channel labels (backchannel, interrupt, planning, tool calls) are merged at the chunk grid.

3.2 Mixture proportions

The corpus is a mixture of seven task families across two stages of training, jointly chosen so that the backbone preserves its world knowledge and language ability, internalises the chunked dual-stream three-channel format, and learns tight time alignment between audio and the action channel. Continued pretraining (CPT) is dominated by ordinary duplex dialogue, augmented with substantial dual-side ASR supervision and general text data. Post-training is much smaller in volume, but is concentrated on the timing-sensitive and action-emitting behaviours that DuplexSLA is built to deliver: turn-taking control cases (pause, interrupt, and backchannel), and three styles of tool-call data (single-action turn-taking, multi-action turn-taking, and backchannel-action). The proportion split is shown in Figure 5, and the concrete scales are listed in Table 2.

Data family	Scale
Continued pretraining (∼500 k hours audio + ∼1.92 M text samples).	
Text	∼1.92 M samples
Duplex dialogue	∼320 k hours
User-channel ASR	∼90 k hours
Assistant-channel ASR	∼90 k hours
Post-training (∼50 k hours).	
Interrupt + backchannel + pause	∼36 k hours
Tool-call (BC-action, single-action, and multi-action)	∼14 k hours

Table 2: Training-data mixture across the two stages. CPT preserves language ability and teaches the dual-stream three-channel format; post-training concentrates on the capability-critical interaction-control and tool-call slices reported in § 5.

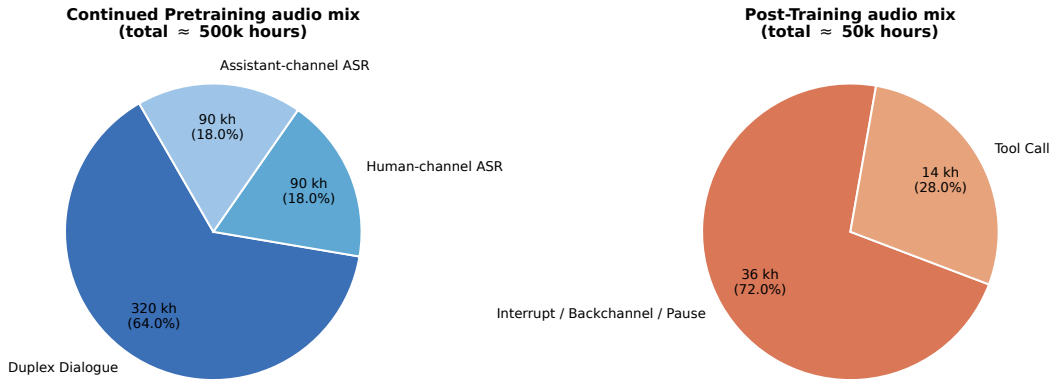

Figure 5: Audio-data distribution across continued pretraining (left) and post-training (right). CPT is dominated by duplex dialogue, with substantial dual-side ASR supervision; post-training is dominated by interaction-control data, with a smaller but capability-critical tool-call slice.

3.3 Action objects on the timeline

Each action object is anchored to its semantic trigger time on the conversational clock. At training time, these offsets are snapped to chunks and emitted on the action channel of the corresponding chunk. Because every chunk has a hard ≤ 10-token action-channel budget (§ 2.3), short bursts of actions cannot always fit into the chunk where they are triggered. The data-construction format therefore turns the action stream into a single FIFO queue keyed by trigger time, with two consistent rules:

- **Within a chunk.** If two or more actions are triggered in the same chunk, they are serialized in trigger-time order (ties broken by id) and concatenated head-to-tail on that chunk's action channel. The chunk-terminating `<|action_end|>` marker is emitted *only* after the last queued action has fully closed, so an open `<|toolcall_begin|>...<|toolcall_end|>` block is never split by an early `<|action_end|>`.
- **Across chunks.** If an action's planning text plus tool-call body exceeds the per-chunk ≤ 10-token budget, the surplus tokens spill into the action segments of the following chunks. Any later-triggered action waits in the queue until the in-flight action has fully drained, and then starts emitting from the next available chunk: it never preempts an earlier action, and never breaks an open `<|toolcall_begin|>...<|toolcall_end|>` block.

Concretely, the within-chunk rule produces concatenated action segments of the schematic form below (shown with two queued actions before any chunk-level truncation is applied):

```
planning_1
<|toolcall_begin|>{"function": "function_name_1", "arguments": "arguments_1"}<|toolcall_end|>
planning_2
<|toolcall_begin|>{"function": "function_name_2", "arguments": "arguments_2"}<|toolcall_end|>
<|action_end|>
```

Each `<|toolcall_begin|>`/`<|toolcall_end|>` block stays atomic; the chunk-terminating `<|action_end|>` appears only after the very last action in the queue has been written out,

and any tokens that overshoot the per-chunk budget are deferred to the following chunks under the across-chunk rule above.

Because the assistant TA4 channel has its own per-chunk token budget, this FIFO queue on the action channel never blocks assistant speech: while the queue is draining over several chunks, the TA4 stream keeps producing audio in lockstep. This is what allows the same backbone to learn "what to do" and "when to do it" from the same supervision target, and is also why the multi-action tool calls in Figure 3b are emitted in the order of the user's request without delaying the spoken response.

3.4 Dual-side ASR is required for time alignment

A subtle but important point is that the action channel learns *both* a delayed user transcript and a delayed assistant transcript. Inside the assistant TA4 layout, the text anchor T is left-aligned within the chunk and does not carry exact timing. By forcing the action channel to emit the assistant transcript at the chunk where each character is actually being spoken, we explicitly tie assistant audio to action time. As a result, the model's internal time clock stays consistent across user audio, assistant audio, and action emission, which is what makes sub-second tool-call latency feasible.

4 Training Recipe

DuplexSLA is initialized from **Step-Audio 2 mini** [20], a 7B-scale audio language model. Training proceeds in two stages: continued pretraining (CPT) brings the backbone into the chunked dual-stream three-channel format, and post-training instils the four duplex-interaction behaviours and the three tool-call patterns evaluated in § 5.

4.1 Stage 1: continued pretraining

The goal of CPT is to make the backbone fluent in the new serialization. The model has to learn three new things at once: (1) the chunk-level interleaving of user audio, assistant TA4, and action text; (2) the strict time alignment between assistant audio and action text via dual-side ASR; and (3) silence behaviours on both the assistant TA4 anchor (`<vad_silence>`, `<tts_pad>`) and the action channel.

The CPT mixture is dominated by audio data ($\sim$ 500k hours in total), supplemented with general text data to preserve world knowledge and reasoning ability:

- Duplex dialogue ($\sim$ 320k h) supplies the base distribution of full-duplex spoken interaction.
- User- and assistant-channel ASR ($2 \times$ 90k h) supply explicit time alignment between audio and action text.
- Text ($\sim$ 1.92M samples) preserves language and reasoning ability while the speech format changes.

After CPT, the model becomes comfortable with the duplex serialization, but does not yet exhibit the targeted real-time interaction behaviours, especially when the user pauses, interrupts, or issues short backchannel feedback.

4.2 Stage 2: capability-oriented post-training

Post-training shifts the data distribution from generic duplex dialogue toward the behaviours we want to evaluate. The mixture is deliberately small ($\sim$ 50k hours), but each slice is highly informative. Two families dominate (post-training block of Table 2):

- Interrupt, backchannel, and pause data ($\sim$ 36k h) drive the action channel to emit the right control labels at the right time, and to switch the assistant TA4 to silence within a small chunk-level latency under interruption.
- Tool-call data ($\sim$ 14k h) drives the model to emit planning text plus structured tool calls on the action channel, both in standard turn-taking single- and multi-action requests and in topically unrelated backchannel-action requests that must not break the assistant's spoken thread.

4.3 Loss with full-duplex-aware masking and reweighting

The training objective is standard next-token cross-entropy on the assistant TA4 stream and the action channel, plus a general text-modelling term on the text-only data slice. On top of this base loss, we apply additional loss masks and per-token weights to selected state tokens and to specific positions in the chunked dual-stream three-channel sequence, so that the optimisation is better matched to the full-duplex training setting (e.g., silence anchors, channel-boundary markers, and task-conditioned segments are not trained as ordinary content tokens). The decoder interface itself does not change between stages or task families, which is what allows DuplexSLA to absorb very different supervision signals, such as duplex dialogue, dual-side ASR, timing control, and tool calls, within a single model.

4.4 Stage division by capability

A turn-based agent can be improved by adding more text or more tool examples. A duplex spoken agent carries the additional burden of *timing*. The CPT stage therefore establishes the timing prior using ordinary duplex dialogue plus dual-side ASR, and the post-training stage sharpens it for pause, interrupt, backchannel, and tool calling. This division was the most data-efficient setup in our experiments: pure duplex dialogue alone teaches turn taking but not interaction control, while starting with capability-heavy data without first stabilizing the duplex serialization leads to noticeably worse speech smoothness on the assistant audio.

5 Evaluation

5.1 Benchmark and evaluation protocol

Existing duplex benchmarks measure pause, interruption, and backchannel behaviour [29], but none of them jointly stress sub-second yielding under semantic interruption, backchannel detection inside the action channel, backchannel-triggered tool calling, and multi-action tool calling on a duplex timeline. We therefore evaluate DuplexSLA on **DuplexSLA-Bench**, a duplex benchmark with 1,200 turn-taking cases (300 each for `normal`, `pause`, `interrupt`, `backchannel`) plus a 900-case tool-call subset (300 each for single-action, multi-action, and backchannel-action requests). All numbers below are reported on this benchmark.

For every turn-taking scenario we report two quantities: an accuracy (did the model behave correctly at all?) and a delay (how far the realised action time was from the desired semantic anchor, computed only on hits). The accuracy windows and delay anchors are summarised in Table 3.

Scenario	Accuracy window	Delay definition
`normal`	assistant speech onset $\in [t_{\text{ue}} - 0.2, +\infty)$	$\lvert t_{\text{speak}} - t_{\text{ue}} \rvert$
`pause`	same as `normal`, but on hesitation-rich user audio	$\lvert t_{\text{speak}} - t_{\text{ue}} \rvert$
`interrupt`	assistant stop time $\in [t_{\text{int}} - 1,\ t_{\text{int}} + 2]$	$\lvert t_{\text{stop}} - t_{\text{int}} \rvert$
`backchannel`	a stop-or-restart event $\in [t_{\text{bc-s}} - 0.2,\ t_{\text{bc-e}} + 2]$	$\lvert t_{\text{label}} - t_{\text{bc-e}} \rvert$

Table 3: Accuracy windows and delay definitions for the four turn-taking scenarios. t_{ue}: user-end time; t_{int}: semantic interrupt anchor; $t_{\text{bc-s}}, t_{\text{bc-e}}$: start and end of the user backchannel utterance.

Each test case is a duplex audio session with semantic anchor times annotated. The system under test is fed in chunked streaming mode, and the assistant audio output is post-processed by an external VAD to obtain speak and stop transitions; for DuplexSLA the action channel is also read directly, so that backchannel labels are evaluated even when there is no user-perceptible audio change. Table 4 states the full procedure in set-theoretic form.

Stage	**Definition**
Inputs	Streaming model $\mathcal{S}$; test case $(\mathbf{u}, \mathcal{H}, s, \mathcal{A})$ with user audio $\mathbf{u}$, dialogue history $\mathcal{H}$, scenario $s \in \{\texttt{normal}, \texttt{pause}, \texttt{interrupt}, \texttt{backchannel}\}$ and anchor set $\mathcal{A}$; prefill flag $p \in \{0, 1\}$ for the context-prefill protocol.
1. Init	$\mathcal{S} \leftarrow \textsc{reset}$. If $p = 1$, $\mathcal{S} \leftarrow \textsc{prefill}(\mathcal{H})$. Initialise event log $E \leftarrow \varnothing$.
2. Stream	Split $\mathbf{u}$ into K chunks of 160 ms. For $k = 0, \ldots, K-1$: $o_k = \mathcal{S}.\textsc{step}(u_k)$, $t_k = 0.16\,k$. Append $(t_k, \texttt{speak}, \varnothing)$ when o_k emits assistant speech, else $(t_k, \texttt{stop}, \varnothing)$; and for each label $\ell \in \textsc{labels}(o_k.\text{act})$ append $(t_k, \ell, o_k.\text{act})$ to E.
3. Score	Let W_s and anchor $t_s^\star$ be given by Table 3, and let $\tau_s = \texttt{speak}$ for $s \in \{\texttt{normal}, \texttt{pause}\}$, $\tau_s = \texttt{stop}$ for $s = \texttt{interrupt}$, and $\tau_s = \texttt{backchannel}$ for $s = \texttt{backchannel}$. Define $\hat{e}_s = \arg\min\{\, t : (t, \tau_s, \cdot) \in E,\ t \in W_s \,\}$; then $\textsc{acc} = \mathbb{1}[\hat{e}_s \text{ exists}]$ and $\textsc{delay} = \lvert \hat{e}_s.t - t_s^\star \rvert$ whenever $\textsc{acc} = 1$. For $s = \texttt{backchannel}$, accuracy is relaxed to any $\{\texttt{stop}, \texttt{speak}\}$ event inside W_s, since closed-source baselines emit no backchannel label and DELAY is therefore reported only when one is present.

Table 4: Pseudocode in tabular, set-theoretic form for the duplex turn-taking evaluation on DuplexSLA-Bench. The three deterministic stages – initialise, stream, score – are shared across systems; only the streaming input and how action labels are read differ. Systems that cannot expose action labels fall back to audio-only events, which is why DELAY on `backchannel` is N/A for non-DuplexSLA baselines (Table 6).

5.2 Tool-call results

Each tool-call test case is a context plus a user request that requires one or more functions. Both systems are run under the same streaming protocol: DuplexSLA reads tool calls directly off the action channel, while the ASR + LLM cascade emits them in a post-ASR planning step. A predicted tool call counts as correct when (1) every ground-truth action has a predicted action with the same function name; (2) the arguments match – exact match, both empty, or judged semantically consistent by an LLM with no "core information conflict";

and (3) the trigger time is legal – not earlier than the ground-truth offset by more than 1.0 s, and not later than the end of the audio by more than 3.0 s. Accuracy is the fraction of cases in which all ground-truth actions are matched, and delay is the average gap on matched actions.

Table 5 reports accuracy and delay across the three call patterns. DuplexSLA matches the cascade in accuracy while delivering $\sim$ 4x lower tool-call delay on average.

Model	Avg. (3 patterns)		Single action		Multi actions		Backchannel action	
	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)
ASR + LLM sys	91.33	2.77	89.33	2.33	89.33	4.71	95.33	1.27
DuplexSLA	85.56	**0.64**	85.67	**0.67**	75.00	**0.68**	96.00	**0.57**

Table 5: Tool-call results on the DuplexSLA-Bench tool-call subset (900 cases). The cascade baseline replaces DuplexSLA with a streaming ASR module whose transcript is fed to an LLM that emits tool calls. DuplexSLA is competitive in accuracy and $\sim$ 4x faster in tool-call delay on average.

5.3 Full-duplex turn-taking results

We evaluate full-duplex behaviour in two settings that mirror typical deployments.

(1) Context prefill. Systems that can prefill the entire dialogue history are evaluated on all four scenarios – `normal`, `pause`, `interrupt`, `backchannel` – alongside DuplexSLA. Table 6 reports the results. DuplexSLA is the only system that handles backchannel correctly (98.33% vs. $\leq$ 40% for all baselines), and it achieves the lowest delay in every scenario. The backchannel delay column is *N/A* for the closed-source baselines because they expose no backchannel label, so the only audio-derived signal is the absence of an interruption.

Model	`normal`		`pause`		`interrupt`		`backchannel`	
	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)
DuplexSLA	**96.00**	**0.27**	**93.33**	**0.27**	**99.33**	**0.40**	**98.33**	**0.32**
gemini-3.1-flash-live	93.67	1.18	94.33	1.17	63.67	0.62	40.00	N/A
gpt-realtime-1.5 (semantic-vad-high)	91.33	1.67	90.33	1.68	79.00	0.68	0.33	N/A
gpt-realtime-1.5 (server-vad-40ms)	82.33	0.95	71.00	1.02	77.00	0.72	13.00	N/A

Table 6: Full-duplex turn-taking results in the context-prefill setting. DuplexSLA achieves the highest accuracy and lowest delay in all four scenarios. Closed-source baselines do not expose a backchannel label, so their backchannel delay is N/A.

(2) No context prefill. Many duplex systems cannot cheaply preload long histories, so we also evaluate a no-prefill setting that reduces to the two scenarios all systems support: `normal` and `pause`. Table 7 reports accuracy and delay against open-source duplex backbones [4, 8] and commercial APIs. DuplexSLA again achieves the lowest delay ($\sim$ 0.30 s) while staying among the highest-accuracy systems (94.34% average), and is the only sub-second model with competitive accuracy.

Model	Avg. (2 scenarios)		`normal`		`pause`	
	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)	Acc. (%)	Delay (s)
DuplexSLA	**94.34**	**0.30**	**95.67**	**0.29**	**93.00**	**0.31**
Freeze-Omni	10.67	0.36	10.33	0.40	11.00	0.33
PersonaPlex	22.34	0.47	22.67	0.38	22.00	0.55
MiniCPM-o	82.00	0.61	83.33	0.62	80.67	0.59
gemini-3.1-flash-live	93.17	1.17	93.67	1.16	93.67	1.18
gpt-realtime-1.5 (semantic-vad-high)	96.50	1.57	96.70	1.57	96.30	1.57
gpt-realtime-1.5 (server-vad-40ms)	85.50	0.83	91.30	0.83	79.70	0.83

Table 7: Full-duplex turn-taking results in the no-context-prefill setting. DuplexSLA is the only sub-second system with competitive accuracy; commercial APIs reach high accuracy but pay > 1 s in delay. Open-source duplex backbones without targeted post-training collapse on the `pause` subset, illustrating that pause robustness has to be supervised explicitly.

5.4 Take-aways

Two patterns are consistent across the tables. (1) On turn taking, DuplexSLA delivers sub-second responses in all four scenarios and is the only system that cleanly handles backchannel detection. (2) On tool calling, DuplexSLA matches the cascade in accuracy at $\sim$ 4x lower delay, because the action channel emits planning and tool calls without waiting for a turn boundary or interrupting assistant audio. Together they validate the central design choice – an explicit action channel on top of a duplex backbone, supervised by the data recipe in § 3.

6 Conclusion

In this work, we propose DuplexSLA, a native full-duplex *speech-language-action* foundation model that places listening, speaking, planning, and tool calling on a shared 160 ms chunk timeline. The dual-stream three-channel formulation – continuous user audio, discrete assistant TA4, and a rate-limited textual action channel – lets a single backbone learn (1) semantic-driven interruption, pause, and backchannel without an external semantic VAD, and (2) in-conversation planning and tool calling that runs in parallel with assistant speech. Trained on a duplex-dialogue plus dual-side ASR pretraining mixture and a smaller capability-oriented post-training mixture, DuplexSLA achieves sub-second latency in all four turn-taking scenarios on the proposed DuplexSLA-Bench, while remaining competitive with an ASR + LLM cascade on tool-call accuracy at 3–4x lower latency. We view DuplexSLA as a step toward duplex spoken agents that combine fluent speech with timely action, and we expect the action-channel design to extend naturally to richer planning signals, multi-turn agentic workflows, and broader open-domain spoken tool use.

References

[1] Peng Wang, Songshuo Lu, Yaohua Tang, Sijie Yan, Wei Xia, and Yuanjun Xiong. A full-duplex speech dialogue scheme based on large language model. *arXiv preprint arXiv:2405.19487*, 2024.

[2] Bandhav Veluri, Benjamin N Peloquin, Bokai Yu, Hongyu Gong, and Shyamnath Gollakota. Beyond turn-based interfaces: Synchronous llms as full-duplex dialogue agents. *arXiv preprint arXiv:2409.15594*, 2024.

[3] Alexandre Defossez, Laurent Mazare, Manu Orsini, Amelie Royer, Patrick Perez, Herve Jegou, Edouard Grave, and Neil Zeghidour. Moshi: a speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024.

[4] Xiong Wang, Yangze Li, Chaoyou Fu, Yunhang Shen, Lei Xie, Ke Li, Xing Sun, and Long Ma. Freeze-omni: A smart and low latency speech-to-speech dialogue model with frozen llm. *arXiv preprint arXiv:2411.00774*, 2024.

[5] Wenyi Yu, Siyin Wang, Xiaoyu Yang, Xianzhao Chen, Xiaohai Tian, Jun Zhang, Guangzhi Sun, Lu Lu, et al. Salmonn-omni: A codec-free llm for full-duplex speech understanding and generation. *arXiv preprint arXiv:2411.18138*, 2024.

[6] Ke Hu, Ehsan Hosseini-Asl, Chen Chen, Edresson Casanova, Subhankar Ghosh, Piotr Zelasko, Zhehuai Chen, Jason Li, Jagadeesh Balam, and Boris Ginsburg. Efficient and direct duplex modeling for speech-to-speech language model. In *Interspeech 2025*, pages 2715–2719, 2025. doi:10.21437/Interspeech.2025-874.

[7] Wenfu Wang, Chenxing Li, Liqiang Zhang, Yiyang Zhao, Yuxiang Zou, Hanzhao Li, Mingyu Cui, Hao Zhang, et al. Covo-audio technical report. *arXiv preprint arXiv:2602.09823*, 2026.

[8] Rajarshi Roy, Jonathan Raiman, Sang-gil Lee, Teodor-Dumitru Ene, Robert Kirby, Sungwon Kim, Jaehyeon Kim, and Bryan Catanzaro. Personaplex: Voice and role control for full duplex conversational speech models. *arXiv preprint arXiv:2602.06053*, 2026.

[9] Zhifei Xie and Changqiao Wu. Mini-omni: Language models can hear, talk while thinking in streaming. *arXiv preprint arXiv:2408.16725*, 2024.

[10] Zhifei Xie and Changqiao Wu. Mini-omni2: Towards open-source gpt-4o with vision, speech and duplex capabilities. *arXiv preprint arXiv:2410.11190*, 2024.

[11] Qingkai Fang, Shoutao Guo, Yan Zhou, Zhengrui Ma, Shaolei Zhang, and Yang Feng. Llama-omni: Seamless speech interaction with large language models. *arXiv preprint arXiv:2409.06666*, 2024.

[12] Qinglin Zhang, Luyao Cheng, Chong Deng, Qian Chen, Wen Wang, Siqi Zheng, Jiaqing Liu, Hai Yu, and Chaohong Tan. Omniflatten: An end-to-end gpt model for seamless voice conversation. *arXiv preprint arXiv:2410.17799*, 2024.

[13] Tu Anh Nguyen, Benjamin Muller, Bokai Yu, Marta R. Costa-jussà, Maha Elbayad, Sravya Popuri, Paul-Ambroise Duquenne, Robin Algayres, Ruslan Mavlyutov, Itai Gat, et al. Spirit lm: Interleaved spoken and written language model. *Transactions of the Association for Computational Linguistics*, 2025. arXiv:2402.05755.

[14] Yemin Shi, Yu Shu, Siwei Dong, Guangyi Liu, Jaward Sesay, Jingwen Li, and Zhiting Hu. Voila: Voice-language foundation models for real-time autonomous interaction and voice role-play. *arXiv preprint arXiv:2505.02707*, 2025.

[15] Donghang Wu, Haoyang Zhang, Chen Chen, Tianyu Zhang, Fei Tian, Xuerui Yang, Gang Yu, Hexin Liu, Nana Hou, Yuchen Hu, et al. Chronological thinking in full-duplex spoken dialogue language models. *arXiv preprint arXiv:2510.05150*, 2025.

[16] Donghang Wu, Haoyang Zhang, Jun Chen, Hexin Liu, Eng Siong Chng, Fei Tian, Xuerui Yang, Xiangyu Zhang, Daxin Jiang, Gang Yu, et al. Mind-paced speaking: A dual-brain approach to real-time reasoning in spoken language models. *arXiv preprint arXiv:2510.09592*, 2025.

[17] Qwen Team et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2(3), 2024.

[18] Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, Xipin Wei, Zhifang Guo, Yichong Leng, Yuanjun Lv, Jinzheng He, Junyang Lin, et al. Qwen2-audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.

[19] Ailin Huang, Boyong Wu, Bruce Wang, Chao Yan, Chen Hu, Chengli Feng, Fei Tian, Jingbei Li, et al. Step-audio: Unified understanding and generation in intelligent speech interaction. *arXiv preprint arXiv:2502.11946*, 2025.

[20] Boyong Wu, Chao Yan, Chen Hu, Cheng Yi, Chengli Feng, Fei Tian, Feiyu Shen, Gang Yu, Haoyang Zhang, Jingbei Li, et al. Step-audio 2 technical report. *arXiv preprint arXiv:2507.16632*, 2025.

[21] Fei Tian, Xiangyu Tony Zhang, Yuxin Zhang, Haoyang Zhang, Yuxin Li, Daijiao Liu, Yayue Deng, Donghang Wu, et al. Step-audio-r1 technical report. *arXiv preprint arXiv:2511.15848*, 2025.

[22] Yuxin Zhang, Xiangyu Tony Zhang, Daijiao Liu, Fei Tian, Yayue Deng, Jun Chen, Qingjian Lin, Haoyang Zhang, Yuxin Li, Jinglan Gong, Yechang Huang, Liang Zhao, Chengyuan Yao, Hexin Liu, Eng Siong Chng, Xuerui Yang, Gang Yu, Xiangyu Zhang, and Daxin Jiang. Step-audio-r1.5 technical report. *arXiv preprint arXiv:2604.25719*, 2026.

[23] OpenAI. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

[24] Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfa Zhu, et al. Qwen3-omni technical report. *arXiv preprint arXiv:2509.17765*, 2025.

[25] Aohan Zeng, Zhengxiao Du, Mingdao Liu, Kedong Wang, Shengmin Jiang, Lei Zhao, Yuxiao Dong, and Jie Tang. Glm-4-voice: Towards intelligent and human-like end-to-end spoken chatbot. *arXiv preprint arXiv:2412.02612*, 2024.

[26] Chaoyou Fu, Haojia Lin, Xiong Wang, Yi-Fan Zhang, Yunhang Shen, Xiaoyu Liu, Haoyu Cao, Zuwei Long, Heting Gao, Ke Li, et al. Vita-1.5: Towards gpt-4o level real-time vision and speech interaction. *arXiv preprint arXiv:2501.01957*, 2025.

[27] Zuwei Long, Yunhang Shen, Chaoyou Fu, Heting Gao, Lijiang Li, Peixian Chen, Mengdan Zhang, Hang Shao, Jian Li, Jinlong Peng, Haoyu Cao, Ke Li, Rongrong Ji, and Xing Sun. Vita-audio: Fast interleaved cross-modal token generation for efficient large speech-language model. *arXiv preprint arXiv:2505.03739*, 2025.

[28] OpenBMB Team. Minicpm-o 2.6: A gemini 2.5 flash level mllm for vision, speech, and full-duplex multimodal live streaming on your phone. https://github.com/OpenBMB/MiniCPM-o, 2025. Accessed: 2025.

[29] Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H Liu, and Hung-yi Lee. Full-duplex-bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities. *arXiv preprint arXiv:2503.04721*, 2025.

[30] Jian Zhang, Linhao Zhang, Bokai Lei, Chuhan Wu, Wei Jia, and Xiao Zhou. Wildspeech-bench: Benchmarking audio llms in natural speech conversation. *arXiv preprint arXiv:2506.21875*, 2025.

[31] Dingdong Wang, Jincenzi Wu, Junan Li, Dongchao Yang, Xueyuan Chen, Tianhua Zhang, and Helen Meng. Mmsu: A massive multi-task spoken language understanding and reasoning benchmark. *arXiv preprint arXiv:2506.04779*, 2025.

[32] S Sakshi, Utkarsh Tyagi, Sonal Kumar, Ashish Seth, Ramaneswaran Selvakumar, Oriol Nieto, Ramani Duraiswami, Sreyan Ghosh, and Dinesh Manocha. Mmau: A massive multi-task audio understanding and reasoning benchmark. *arXiv preprint arXiv:2410.19168*, 2024.

[33] Yayue Deng, Guoqiang Hu, Haiyang Sun, Xiangyu Zhang, Haoyang Zhang, Fei Tian, Xuerui Yang, Gang Yu, and Eng Siong Chng. Multi-bench: A multi-turn interactive benchmark for assessing emotional intelligence ability of spoken dialogue models. *arXiv preprint arXiv:2511.00850*, 2025.

[34] Siddhant Arora, Zhiyun Lu, Chung-Cheng Chiu, Ruoming Pang, and Shinji Watanabe. Talking turns: Benchmarking audio foundation models on turn-taking dynamics. In *The Thirteenth International Conference on Learning Representations*, 2025. arXiv:2503.01174.

[35] Yiming Chen, Xianghu Yue, Chen Zhang, Xiaoxue Gao, Robby T. Tan, and Haizhou Li. Voicebench: Benchmarking llm-based voice assistants. *arXiv preprint arXiv:2410.17196*, 2024.

[36] Qian Yang, Jin Xu, Wenrui Liu, Yunfei Chu, Ziyue Jiang, Xiaohuan Zhou, Yichong Leng, Yuanjun Lv, Zhou Zhao, Chang Zhou, and Jingren Zhou. Air-bench: Benchmarking large audio-language models via generative comprehension. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.07729.

[37] Junyi Ao, Yuancheng Wang, Xiaohai Tian, Dekun Chen, Jun Zhang, Lu Lu, Yuxuan Wang, Haizhou Li, and Zhizheng Wu. Sd-eval: A benchmark dataset for spoken dialogue understanding beyond words. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.13340.

[38] Ruiqi Yan, Xiquan Li, Wenxi Chen, Zhikang Niu, Chen Yang, Ziyang Ma, Kai Yu, and Xie Chen. Uro-bench: Towards comprehensive evaluation for end-to-end spoken dialogue models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. arXiv:2502.17810.

[39] Heyang Liu, Yuhao Wang, Ziyang Cheng, Ronghua Wu, Qunshan Gu, Yanfeng Wang, and Yu Wang. Vocalbench: Benchmarking the vocal conversational abilities for speech interaction models. *arXiv preprint arXiv:2505.15727*, 2025.

Appendix

A Per-Chunk Serialization Case Studies

This appendix gives concrete chunk-by-chunk traces of the dual-stream three-channel format introduced in § 2. For readability, each row corresponds to one chunk (160 ms), the user audio segment is abbreviated as "$U\,U$", and the TA4 unit is shown as "$T(\cdot)\,A\,A\,A\,A$", where T holds either a text token (e.g., "确"), the silence anchor `<vad_silence>`, or the trailing pad `<tts_pad>`. The action segment, when not empty, is shown after the assistant TA4 and before the chunk-terminating `<|action_end|>`.

A.1 User-channel ASR

The user-channel transcript is emitted on the action channel with a fixed lag of 2 chunks (320 ms). Tokens that fall in the same chunk are merged.

```
<|user_audio_begin|> U U(今天) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>)
A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(天气很) <|user_audio_end|> <|assistant_audio_begin|>
T(<vad_silence>) A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(好) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A
A A A <|assistant_audio_end|> 今天 <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> 天气很 <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(确) A A A A
<|assistant_audio_end|> 好 <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(实) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<tts_pad>) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> <|action_end|>
```

A.2 Assistant-channel ASR

Necessity of assistant-side ASR. The action channel cannot simply delay-copy the T tokens of the assistant TA4 by two chunks. The TA4 layout looks like

$$TA_4\ \ TA_4\ \ TA_4\ \ TA_4\ \ T(\texttt{<tts\_pad>})A_4\ \ T(\texttt{<tts\_pad>})A_4 \ldots$$

That is, the assistant text tokens are *left-aligned* inside the chunk and do not carry exact timing: a single Chinese word can be packed into the first T slot of a chunk while the corresponding audio is actually played in the next chunk. Re-emitting the assistant transcript on the action channel at the chunk where each character is genuinely being spoken therefore teaches DuplexSLA the correct time alignment between assistant audio and action time.

```
<|user_audio_begin|> U U(今天) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>)
A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(天气很) <|user_audio_end|> <|assistant_audio_begin|>
T(<vad_silence>) A A A A <|assistant_audio_end|> <|action_end|>
```

```
<|user_audio_begin|> U U(好) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A
A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(确) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(实) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<tts_pad>) A A A A
<|assistant_audio_end|> 确 <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> 实 <|action_end|>
```

A.3 Duplex dialogue with planning + tool call

The user comments that the car is cold; once the request becomes semantically clear, the action channel emits a planning fragment and a tool call (`set_car_setting`). The assistant audio remains coherent across the same chunks. Because of the per-chunk action-token cap ($\leq$ 10 tokens), the planning text and the tool-call body span several consecutive chunks rather than a single one, following the FIFO rule in § 3.3. The strict time-alignment prior installed by dual-side ASR during CPT (§ 4) lets the model treat this multi-chunk spillover as a bounded, budget-induced transmission delay rather than as drift in the offset itself: the first action token still emerges at the chunk that owns the semantic anchor, so the realised trigger time remains aligned with the annotated offset.

```
<|user_audio_begin|> U U(车里) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>)
A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(真冷) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>)
A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(啊) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A
A A A <|assistant_audio_end|> 用户感到有点冷，应该打开空调开暖 <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> 风，调整到合适的温度
<|toolcall_begin|>{"function": "set_car_setting", <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(我) A A A A
<|assistant_audio_end|> "arguments": "AC=26 度"}<|toolcall_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(明白) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(了) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(这就) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(帮) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(你) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(打开) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(空调) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<tts_pad>) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<tts_pad>) A A A A
<|assistant_audio_end|> <|action_end|>
```

```
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<tts_pad>) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A A A
A <|assistant_audio_end|> <|action_end|>
```

A.4 An illustrative interrupt case

The user starts to follow up on the assistant's previous answer ("you are right") and then immediately rolls into a real new statement ("but the project schedule is tight, I don't really have a choice"). DuplexSLA is expected to keep speaking while the user is only acknowledging, and then yield within a small chunk-level latency once the new content starts. The action channel emits an `interrupt` label, and the assistant TA4 immediately switches from text and audio tokens to silence anchors. This trace mirrors the right-hand panel of Figure 2.

```
<|user_audio_begin|> U U(你说) <|user_audio_end|> <|assistant_audio_begin|> T(身体) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(得对) <|user_audio_end|> <|assistant_audio_begin|> T(健康) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(是) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(但项) <|user_audio_end|> <|assistant_audio_begin|> T(最) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(目很) <|user_audio_end|> <|assistant_audio_begin|> T(重要) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(紧) <|user_audio_end|> <|assistant_audio_begin|> T(的) A A A A
<|assistant_audio_end|> 检测到用户插话 <interrupt> <|action_end|>
<|user_audio_begin|> U U(我也) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>)
A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(没办) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>)
A A A A <|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(法) <|user_audio_end|> <|assistant_audio_begin|> T(<vad_silence>) A
A A A <|assistant_audio_end|> <|action_end|>
```

The example also shows that the same acknowledgement ("you are right"), in the backchannel scenario, would be followed by user silence rather than a new statement; in that case DuplexSLA emits a `backchannel` label on the action channel and continues speaking.

A.5 An illustrative backchannel case

In this case, the assistant is in the middle of a long answer; the user inserts a brief "没错 / that's right" acknowledgement and then stays silent. The expected behaviour is that the action channel emits a `backchannel` label inside the user-utterance window, while the assistant TA4 keeps producing the planned response. This mirrors the left-hand panel of Figure 2.

```
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(相依为) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(没) <|user_audio_end|> <|assistant_audio_begin|> T(命的) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U(错) <|user_audio_end|> <|assistant_audio_begin|> T(感觉) A A A A
<|assistant_audio_end|> 检测到附和语气 <backchannel> <|action_end|>
```

```
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(比) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(直接) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(撒糖) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(有意思) A A A A
<|assistant_audio_end|> <|action_end|>
<|user_audio_begin|> U U <|user_audio_end|> <|assistant_audio_begin|> T(多了) A A A A
<|assistant_audio_end|> <|action_end|>
```

B Action vocabulary and task-conditioned system prompts

Beyond textual planning and tool-call JSON, the action channel uses a small set of canonical control-label phrases. During data construction, the same `name` field is paraphrased by several near-synonyms so that the model is not over-fit to a single surface form. Table 8 lists the canonical name and representative paraphrases observed in the training data, and Table 9 summarises the per-task system prompt used during training, where the assistant-speaker placeholder $\{\cdot\}$ is filled with one of the 18 main voice-clone speakers.

Action `name`	Trigger context	Representative canonical phrases (Chinese, with paraphrases)
`response`	user finishes a turn	用户发言结束 / 检测到表达完毕 / 接收到完整内容
`interrupt`	user starts a real new thought during assistant speech	检测到用户插话 / 识别到插话意图 / 检测到有效发言
`backchannel`	user emits short feedback without taking the floor	检测到附和语气 / 识别到轻微反馈 / 用户仅做确认
`asr`	duplex ASR supervision (Appendix A.1, A.2)	no canonical phrase; planning text is the delayed transcript token
tool name	tool-use scenario	free planning text plus structured JSON: `<|toolcall_begin|>{"function": `f`, "arguments": `θ`}<|toolcall_end|>`

Table 8: Canonical control labels used on the action channel. The label set is kept compact so that turn-taking decisions are decoupled from spoken content.

Task family	Training-time system prompt
`dialogue`	(empty)
`asr_human`	请记录下你所听到的语音内容，只记录用户说的内容。
`asr_assistant`	请记录下你所听到的语音内容，只记录助手说的内容。
`interpret`	请翻译用户说的内容。
`toolcall`	你是一个专注于与人互动的 AI，既能聊天，也能使用工具来解决用户的问题。
`interrupt`	你是一个 AI 语音助手，用 $\{\cdot\}$ 的声音来说话。
`backchannel`	你是一个 AI 语音助手，用 $\{\cdot\}$ 的声音来说话。
`pause`	你是一个 AI 语音助手，用 $\{\cdot\}$ 的声音来说话。

Table 9: Per-task system prompt used during training. The model picks up the task signal from the system prompt; $\{\cdot\}$ is filled at sample-build time with the name of one of the 18 canonical assistant speakers.

C DuplexSLA-Bench composition and tool schema coverage

DuplexSLA-Bench is a benchmark of 2,100 cases used in § 5. It is organised into two subsets that exercise the two highlight capabilities of DuplexSLA. Table 10 summarises the case counts, and Table 11 lists the full set of 50 tool schemas exercised by the tool-call subset, grouped by intent family.

Subset	#cases	Notes
*Turn-taking subset (*1,200 *cases, Tables 6 and 7).*		
`normal`	300	ordinary end-of-turn response
`pause`	300	hesitation-rich within-turn silence
`interrupt`	300	semantic interruption mid-assistant-speech
`backchannel`	300	short user feedback without floor transfer
*Tool-call subset (*900 *cases, Table 5).*		
`single-action`	300	one explicit user request, single function
`multi-action`	300	one user turn, multiple ordered functions
`backchannel-action`	300	topically unrelated function triggered while assistant keeps speaking

Table 10: DuplexSLA-Bench composition.

Table 11: Full cabin and smart-home tool schema used during training and evaluation. The schema contains 50 functions, organised into car-cabin control, navigation, media, and broader on-device search and query intents. Backchannel, interrupt, and pause are *not* part of this schema – they are handled by the dedicated control-label vocabulary in Table 8.

Function name	Description
Cabin and hardware control.	
`open_car_setting`	Turn on a car-related hardware feature or software setting (AC, windows, defrost, etc.).
`close_car_setting`	Turn off a car-related hardware feature or software setting.
`set_car_setting`	Set a car-related setting to a target value (AC temperature, seat position, etc.).
`increase_car_setting`	Increase a car-related setting value (raise AC temperature, raise volume, etc.).
`decrease_car_setting`	Decrease a car-related setting value.
`query_car_setting`	Query the current state of a car setting (AC temperature, seat position, etc.).
`save_car_setting`	Save the current car settings for quick recall later.
`set_pet_car_setting`	Configure pet-aware cabin settings (pet mode, climate).
`set_car_alarm`	Configure the car alarm system (alarm time, on or off).
System-level settings and apps.	
`open_system_setting`	Open a system-level settings panel (display, sound, etc.).
`close_system_setting`	Close a system-level settings panel.

(continued on next page)

(continued from previous page)

Function name	Description
`set_system_setting`	Set a system-level value (display brightness, master volume, etc.).
`increase_system_setting`	Increase a system-level value (volume, brightness).
`decrease_system_setting`	Decrease a system-level value (volume, brightness).
`disconnect_system_setting`	Disconnect a system-level connection (Bluetooth, Wi-Fi).
`open_app`	Open an in-vehicle application.
`close_app`	Close an in-vehicle application.
`switch_page`	Switch between UI pages via voice.
`scroll`	Scroll the UI vertically or horizontally.
`select_option`	Select one option from a multi-choice prompt.
Navigation.	
`navigate`	Start route planning and turn-by-turn navigation.
`change_navigation_route`	Change the current navigation route.
`add_waypoint`	Add a waypoint to the current route.
`remove_waypoint`	Remove a waypoint from the current route.
`resume_navigation`	Resume the active navigation session.
`query_arrival_time`	Query the estimated arrival time at the destination.
`query_distance`	Query the distance between two locations.
`query_road_conditions`	Query current road and traffic conditions.
`search_along_route`	Search points of interest along the planned route (gas, restaurants, parking).
Media playback.	
`play_media`	Play a piece of media content (music, podcast, video).
`play_broadcast`	Play radio or broadcast content (radio shows, news).
`play_online_video`	Play an online video.
`search_music`	Search the music library for songs, albums or artists.
`search_online_video`	Search online video platforms for a specific clip.
`next_track`	Advance to the next track or media item.
`previous_track`	Go back to the previous track or media item.
Search and queries.	
`search_food`	Search for food-related information (restaurants, dishes).
`search_entertainment`	Search for entertainment content (movies, music, etc.).
`search_lifestyle`	Search for lifestyle services and information (housekeeping, gyms, events).
`search_shopping`	Search for shopping information (items, stores, deals).
`search_scenic_spot`	Search for tourist attractions (intro, hours, ticket price).
`search_hotel`	Search hotel information (name, location, booking).
`search_travel`	Search travel options (flight, train, public transit).

(continued on next page)

(continued from previous page)

Function name	Description
`search_building`	Search information about buildings (name, location, history).
`search_recognition`	Run a voice-recognition-driven search to identify and look up content.
`query_weather`	Query current or forecast weather.
`query_calendar`	Query calendar information (dates, holidays, events).
`query_stock`	Query stock-market information (price, trend).
`generate_text_resource`	Turn text into a resource (document, report, etc.).
`make_call`	Place a phone call to a contact or to a dialed number.

D Inference and serving notes

The decisions that matter for serving are summarised below; the per-chunk action-token cap is a deployment budget rather than an architectural constant (§ 2.3).

- Conversational clock: $\Delta = 160$ ms.
- Per-chunk model output: 5 assistant TA4 tokens (always) plus up to 10 action text tokens.
- User audio is encoded by a causal speech front end; no future user audio is required to advance one chunk.
- When the action stream needs to emit more than 10 tokens in one chunk (e.g., a long planning text plus a JSON tool-call body), the surplus spills into the action segment of the next chunk. As a result, tool-call closing markers can land in a later chunk than the opening marker, but the trigger time of the action object is always anchored to the chunk where the planning text starts.

E Action object schema

For completeness, each action object on the action channel carries the following conceptual fields. Concrete storage layouts vary across data families, but the abstract schema below is shared by all of them.

- `name`: function name – one of the 50 cabin and smart-home tool schemas, or one of `response`, `interrupt`, `backchannel`, or `asr`.
- `planning`: optional natural-language planning text, kept short so that it fits within a few chunks.
- `parameters`: optional JSON-style argument dictionary; for `asr` or control-only labels this is empty.
- `offset`: semantic trigger time on the conversational clock, snapped to a chunk index at training time.

The action stream is a single FIFO queue keyed by `offset` (§ 3.3): when two or more action objects fall into the same chunk, they are serialized in trigger-time order on the action channel (ties broken by id); when an action's tokens exceed the per-chunk ≤ 10-token budget, the surplus spills into the following chunks while any later-triggered action waits in the queue. This is what allows multi-action tool calls (Figure 3b) to share a short timeline window without losing temporal interpretability or breaking the assistant audio.